Exercise 2 UART Serial Communication & FIFO

Objectives

The goal of this exercise is to implement a USB/UART transmitter and receiver that operate using serial protocol for communication with a PC running Matlab. In addition to this, a FIFO will be implemented and integrated into the design. The exercise is divided into three parts:

Part 1 focuses on the USB/UART receiver, and provides an introduction into the communication protocol.

Part 2 combines the receiver with a transmitter to provide a complete loopback solution to verify the communication path to and from the FPGA.

Part 3 adds the FIFO component to the design. A complete system testbench will include the modules from Parts 1, 2, & 3 to give a high level perspective. Matlab and the Matlab commands required to use virtual communication (ie. USB/COM) ports are introduced.

Part 1: UART Receiver

Use the provided "MLUART RX.vhd" UART receiver module to implement a simple circuit that receives ASCII characters from the PC to the FPGA. Use the LEDs to display the binary representation of the ASCII value of the character (e.g. `0x31` for '1'). The entity for the TX interface is defined in the "Implementation Notes" section.

The baud rate for the provided design is 115,200. Data can be sent from the PC in either Matlab using the commands shown in the Matlab Tutorial ("MatlabTut.pdf"), or using `screen` in a terminal. Connect to the FPGA by opening a new terminal (avoid using the one that Vivado is running from) and running the following command:

```
$ screen /dev/ttyUSB1 115200
```

`/dev/ttyUSB1` is the port the FPGA UART is connected to and `115200` is the baud rate. Typing characters while screen is open will send them to the FPGA, but the characters will not appear in the terminal. Kill the screen session by pressing `ctrl+a`, then `shift+k`, and confirm with `y`.

Simulate the testbench and analyze provided models to learn about the implementation details of this design. Draw RTL diagram showing all components and their interactions - there is no need to specify all the details of state machines or other components (e.g. clock divider will be one component, counter will be one component etc.).

Part 2: PC-to-FPGA UART Loopback

Using the receiver from Part 1 as reference, create a separate transmitter module that will transmit ASCII characters from FPGA to the PC. Then combine the receiver and transmitter modules into a single design that will receive and then retransmit back each character it receives from the PC.

Modify the testbench from Part 1 to simulate the loopback circuit to ensure that any characters sent into the system are transmitted back. Use Matlab or a terminal to test the system on the FPGA.

Parts 1 and 2 must be signed off by 10/5/2026.

Part 3: FIFOs and AXI Protocol

AXI Stream

AXI Stream is a protocol that can be used to interface between two blocks when only data must be sent. For example, when receiving data over UART and sending it into a FIFO, there is no need to specify what address the data should be written to, just that it is valid. AXI Stream functions much like a single channel of the AXI Lite or Full protocols, in that it contains data, ready, and valid signals. The master places the data on the bus and indicates that it is valid. When the slave is prepared to receive data, it signals this by pulling ready high. The transfer takes place when both signals are high. A sample transaction is shown in Figure 2.1.

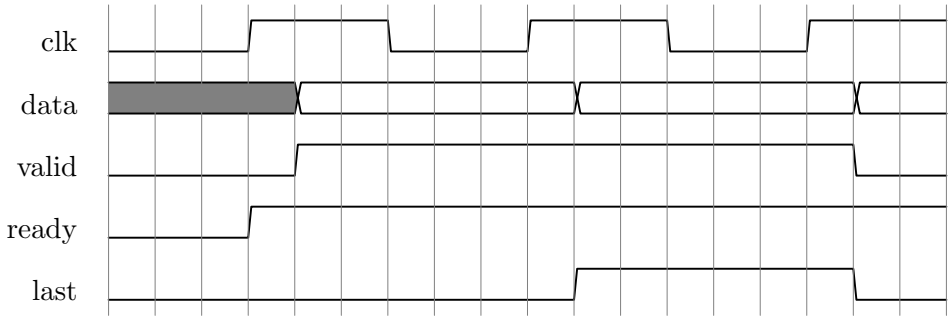


Figure 2.1: Sample AXI Stream Transaction

The stream protocol is bursty in nature, so more than one data word can be sent in a single transaction, though not in a single clock cycle. To allow this, the master and slave should assert ready and valid for as long as possible. Because the data will often be in packets, a last signal is also used to indicate when the end of a group of data is reached. In the example, the packet is two words long, but longer packets are also possible. For further details, the AXI Stream specification is posted on MyCourses.

Design

A diagram of the UART FIFO system is shown in Figure 2.2.

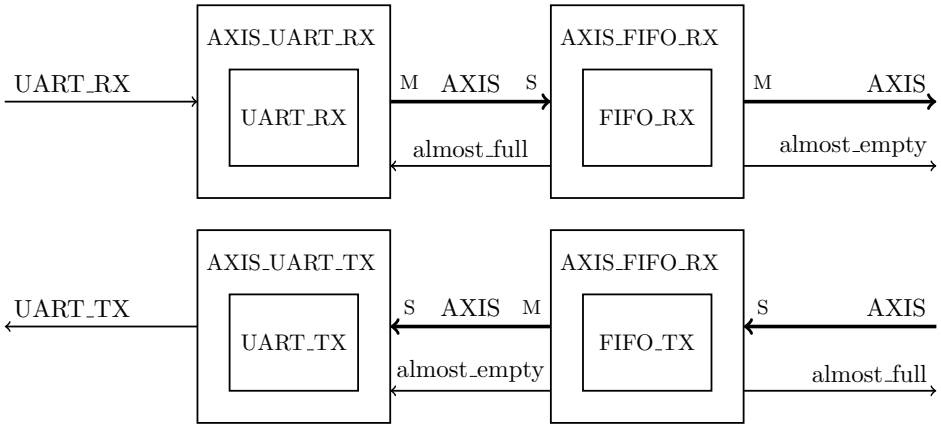


Figure 2.2: Block diagram of UART FIFO AXI Stream system

Create a wrapper for the UART components so that they have AXI Stream interfaces with `data`, `valid`, and `ready` signals.. The RX module should have a master interface and the TX module should have a slave interface. In the UART FIFO application, only one byte is sent at a time, so the `last` signal will be left out and not configured in the FIFO. Also include ports for "fifo_almost_full" and "fifo_almost_empty" signals as shown in Figure 2.2 and include these in the logic for transactions. "Almost full" means a FIFO only has one more write until full and "almost empty" means a FIFO only has one read left until empty.

Create a top level module which instantiates both of the AXIS UART components. Use the IP catalog and search for "AXI4-Stream Data Fifo". Double click on this component to open the IP Customization window. Click on the "Flags" tab and set the "Enable almost full" and "Enable almost epmty" to "Yes". Click "OK" and then "Generate" to add this IP to the project. Use the instantiation template and instantiate two instances (one for RX and one for TX) of this FIFO in the top level module.

Connect the receive UART AXIS interface to the slave interface of the RX FIFO, connect the master interface of the RX FIFO to the slave interface of the TX FIFO, and connect the transmit UART AXIS interface to the master interface of the TX FIFO.

Create a testbench which must send 16 bytes of data and receive it back to verify the same data that went in comes back out.

Before synthesis, double click on "axis_data_fifo_0" under Sources and change the FIFO depth to be largest possible (32,768), leaving the width at 8 bits.

Use Matlab to transmit "Lena.pgm" and ensure that the image that comes back out is identical to the image that was sent in.

Implementation Notes

A UART (Universal Asynchronous Receiver/Transmitter) is a communication module that translates data between parallel and serial forms. The universal designation indicates that the data format and transmission speeds are configurable. Communicating UARTs usually have no shared timing system apart from the communication signal. Typically, UARTs resynchronize their internal clocks on each change of the data line that is not considered a spurious pulse. Obtaining timing information in this manner, they reliably receive when the transmitter is sending at a slightly different speed than it should. In this exercise, the design must sample the incoming signal at a rate of at least 16x the communication speed. The system will be operating at 115,200 baud. In telecommunication and electronics, baud is synonymous with pulses per second therefore this system will be using a data transmission frequency of 115.2 kHz. An example data transmission sequence for reference is shown in Figure 2.3.

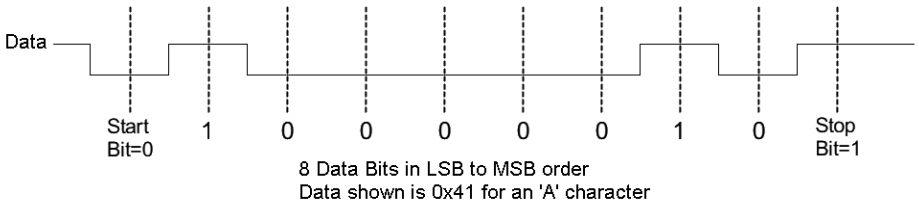


Figure 2.3: Sample data transmission of an 'A' character.

Figure 2.4 provides a table that can be used to convert human readable characters into their ASCII equivalents, which will be transmitted and received by the UARTs.

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040		Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174			|
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177	&#127;	DEL

Source: www.LookupTables.com

Figure 2.4: ASCII table.

The interface for the TX should look something like:

```
entity MLUART_TX is
    port (
        clk_100MHz          : in  std_logic;
        clk_en_16_x_baud    : in  std_logic;
        data_in             : in  std_logic_vector(7 downto 0);
        send_data           : in  std_logic;
        UART_TX             : out std_logic;
        send_data_complete  : out std_logic
   );
end MLUART_TX;
```

Exercise 2: UART Serial Communication & FIFO

Student's Name:_____ Section:_____

Demo		Point Value	Points Earned	Date
Part 1: UART RX	Simulation	5		
	Hardware Demo	5		
	RTL diagram	10		
Part 2: Loopback	State Diagram	10		
	Simulation	10		
	Hardware Demo	10		
Part 3: FIFO	RTL Diagram	10		
	Simulation	10		
	Hardware Demo	10		

To receive any grading credit students must earn points for both the demonstration and the report.

Exercise 2: UART Serial Communication & FIFO

Report		Point Value	Points Earned	Comments
Report	Discussion	20		
Total for prelab, demo, and report		100		

An electronic submission of project files via MyCourses is required. For information on how to properly submit the files, see the document "MyCoursesTut.pdf" on MyCourses.

Required file structure of tarball submission for this exercise:

```
Ex2
├── Part2
│   ├── [all necessary .vhd files]
│   └── Part2.xdc
└── Part3
    ├── [all necessary .vhd files]
    ├── Part3.xdc
    └── Part3.m
```